BusinessIntelligence.ai

Complete Technical Architecture, Analytics Engine & Field Calculation Report

1. Executive Overview & System Architecture

BusinessIntelligence.ai is an enterprise Decision Intelligence product designed to automatically detect, investigate, attribute, and recommend business actions for revenue and KPI anomalies. Unlike standard LLM applications that ask generative AI models to calculate numbers (introducing hallucinations and non-deterministic results), BusinessIntelligence.ai enforces a strict separation between **Deterministic Analytics** and **Generative Narration**.

Core Architecture Principles:

- **SQL & Math Core (DuckDB):** All KPI numbers, time-series aggregations, and variance decompositions are calculated via SQL.
- **Deterministic Attribution:** Price effect and volume effect driver analysis are calculated using closed-form financial math.
- **5-Pillar Statistical Confidence Scoring:** Quantitative confidence score (0-100%) computed from data quality, coverage, driver agreement, evidence relevance, and attribution certainty.
- **Pre-Query Security Interceptor:** Role-based access control (RBAC) evaluates permissions before database queries are constructed.
- **Persona-Adapted Narration:** Generative AI (Google Gemini) receives strictly structured statistics and synthesized evidence to produce C-Suite or Regional Manager briefings.

System Module Layout:

Module Path	Component Name	Function & Responsibility
semantic/kpi_definitions.yaml	Semantic Contract	Single Source of Truth for KPIs, SQL formulas, refresh cadences & RBAC access rules.
security/authorization.py	Security Interceptor	Enforces pre-query region authorization checks before DuckDB execution.
analytics/kpi_engine.py	KPI Calculation Engine	DuckDB SQL query parser comparing 7-day current vs previous period statistics.
analytics/anomaly_detection.py	Anomaly & Sparse Checker	Threshold magnitude evaluation and product history baseline depth check (<14 days).
analytics/driver_analysis.py	Variance Decomposition	Isolates price effects, volume effects, marketing contribution & delivery issue impacts.
retrieval/retriever.py	Semantic Retriever	TF-IDF cosine similarity & Gemini dense embeddings matching logs & tickets.
analytics/impact_calculator.py	Impact & Exposure	Scales 7-day metric changes into monthly exposure (■) and normalized 0-100 impact scores.
analytics/recommendation_engine.py	Guardrailed Rule Engine	Generates actionable recommendations gated by system confidence score levels.
analytics/simulator.py	What-If Simulator	Linear elastic scenario model projecting monthly financial recovery potential.
llm/narrative.py	Persona Narrator	Generates Gemini-powered C-Level / Regional Manager executive summaries with fallback templates.

2. Detailed KPI Calculation Engine & Formulas

The KPI Engine calculates performance metrics by evaluating a rolling 14-day window relative to the dataset's maximum date (max_date):

- **Current Period (P_{curr}):** max_date - 6 days to max_date (7 days inclusive)
- **Previous Period (P_{prev}):** max_date - 13 days to max_date - 7 days (7 days inclusive)

KPI Name	Mathematical Formula	SQL Query Logic
Revenue	$$\sum (Units \times Price)$$	SELECT COALESCE(SUM(units * price), 0) FROM sales WHERE ...
Orders	$$\sum Orders$$	SELECT COALESCE(SUM(orders), 0) FROM sales WHERE ...

ASP (Avg Selling Price)	$$\frac{\sum Revenue}{\sum Units}$$	SELECT SUM(units * price) / SUM(units) FROM sales WHERE ...
Conversion Rate	$$\frac{\sum Orders}{\sum Clicks}$$	SELECT s.orders * 1.0 / m.clicks FROM sales s, marketing m WHERE ...

Percentage Movement Formula:
Change % is calculated as: Change % = ((V_current - V_previous) / V_previous) * 100.
If abs(Change %) >= Threshold (Revenue: 5%, Orders: 5%, ASP: 3%, Conversion: 5%), an anomaly alert is triggered.

3. Driver Decomposition & Variance Analysis Math

For revenue anomalies, the engine decomposes the total revenue change (Total Change = R_curr - R_prev) into isolated underlying drivers:

1. Price Effect: Isolates revenue shift caused strictly by average selling price changes:
Price Effect = (Price_curr - Price_prev) * Units_curr

2. Volume Effect: Isolates revenue shift caused strictly by change in total units sold:
Volume Effect = (Units_curr - Units_prev) * Price_prev

3. Marketing Sub-Decomposition: Attributes portion of volume drop to marketing campaign conversion changes:
Marketing Impact = Conversion_Change_% * Price_prev * abs(Units_curr - Units_prev)

4. Delivery Impact: Evaluates support tickets logged for delivery delays:
Estimated Lost Orders = (Severe Tickets * 1.5) + (Minor Tickets * 0.5)
Delivery Impact = Estimated Lost Orders * Price_prev

5. Contribution Normalization: Converts raw impact into percentages of total revenue change. If unallocated residual variance exists, an *'Other factors'* driver is added to guarantee driver contributions total 100%.

4. The 5-Pillar Statistical Confidence Score Engine

The system confidence score (0 - 100%) quantifies the reliability of the analysis before recommending business actions. It combines 5 weighted base pillars and applies dynamic penalties for stale data or conflicting evidence.

Pillar	Weight	Calculation Logic & Scoring Rules
P1: Data Quality	20%	Base score = 95 if marketing data exists (50 if missing). Subtracts 15 if competitor data is missing. Range: [0, 100].
P2: Historical Coverage	20%	Score = 95 if product history >= 14 days; drops to 30 if history is sparse (< 14 days).
P3: Driver Agreement	20%	Score = 85 if >= 3 aligned drivers identified; 70 if >= 1 driver; 30 if 0 drivers.
P4: Evidence Relevance	20%	Score = 90 if competitor AND support evidence exist; 40 if neither exist; 80 otherwise.
P5: Attribution Certainty	20%	Score = min(abs(Explained Driver Variance) / abs(Total Revenue Change), 1.0) * 100.

Base Score Formula:
Base Score = (0.20 * P1) + (0.20 * P2) + (0.20 * P3) + (0.20 * P4) + (0.20 * P5)

Dynamic Deductions & Penalties:
• **Stale Marketing Data Penalty:** -15% if marketing source age > 168 hours.
• **Stale Competitor Data Penalty:** -15% if competitor source age > 72 hours.
• **Contradiction Penalty:** -20% if conflicting signals detected (e.g. high campaign CTR or positive shipping tickets while delivery backlog is claimed).

Final Confidence Score:
Final Score = max(10, Base Score - Deductions)
• **HIGH Confidence:** Score >= 80% | **MEDIUM Confidence:** 55% <= Score < 80% | **LOW Confidence:** Score < 55%

5. Business Financial Exposure & Impact Calculator

The impact calculator scales weekly period changes to a **monthly financial exposure** (4.33 weeks/month) and computes a normalized 0-100 Impact Score:

Monthly Exposure Math:
• **Revenue:** Monthly Exposure = abs(Current_Val - Prev_Val) * 4.33
• **Orders:** Monthly Exposure = (abs(Orders_diff) * ■35,000 avg ASP) * 4.33
• **ASP:** Monthly Exposure = (abs(ASP_diff) * 300 units/week) * 4.33
• **Conversion Rate:** Monthly Exposure = (abs(CR_diff) * 5,000 clicks * ■35,000 ASP) * 4.33

Normalized Impact Score Formula:

Raw Impact = w_kpi * abs(Change_%) * log10(Monthly_Exposure + 1) * (Confidence_% / 100)
Impact Score = min(round(Raw Impact * 3.5, 1), 100.0)
• *KPI Multipliers (w_kpi):* Revenue = 1.0, Conversion Rate = 0.8, Orders = 0.7, ASP = 0.6.
• *Impact Categories:* **HIGH** (Score >= 60), **MEDIUM** (25 <= Score < 60), **LOW** (Score < 25).
• *Estimated Recovery Potential:* Calculated as 40% (min) to 70% (max) of monthly financial exposure.

6. Confidence Guardrails for Recommendations

The system confidence score strictly gates the risk level of generated business recommendations:

Confidence Level	Guardrail Policy	Recommendation Type
LOW (< 50%)	**Abstain from strategic changes.** High risk of error on incomplete data.	*'Continue monitoring & collect additional data.'* (Risk avoidance)
MEDIUM (50% - 75%)	**Advisory & Pilot testing.** Validate telemetry before scaling spend.	*'Conduct competitive audit & run limited pilot promotions.'*
HIGH (> 75%)	**Full Action Execution.** Clear, high-priority operational response.	*'Review and adjust pricing model in region to match competitor.'*

7. Semantic Evidence Retrieval & Cosine Similarity Math

Supporting logs, competitor announcements, and customer tickets are matched against identified drivers using vector similarity math:

Pure Python TF-IDF Cosine Similarity:
For query *q* and document text *d* over vocabulary *V*:
TF(t, d) = count(t, d) | IDF(t) = ln(1 + N / (1 + DF(t)))
Vector(d)[i] = TF(t_i, d) * IDF(t_i)
Cosine Similarity(q, d) = (v_q . v_d) / (||v_q|| * ||v_d||)

If a Gemini API key is active, dense vector embeddings (models/embedding-001) are computed automatically with TF-IDF as a fallback.

8. What-If Scenario Simulator & Linear Elasticity Model

The scenario simulator models potential revenue recovery if a negative driver is mitigated:
Recovery % = Proposed Driver Impact % - Original Driver Impact %
Monthly Estimated Recovery (■) = (Current Weekly Value * 4.33) * (Recovery % / 100)
All simulation metrics are explicitly tagged as ■ SIMULATED SCENARIO to maintain clean separation from observed historical data.